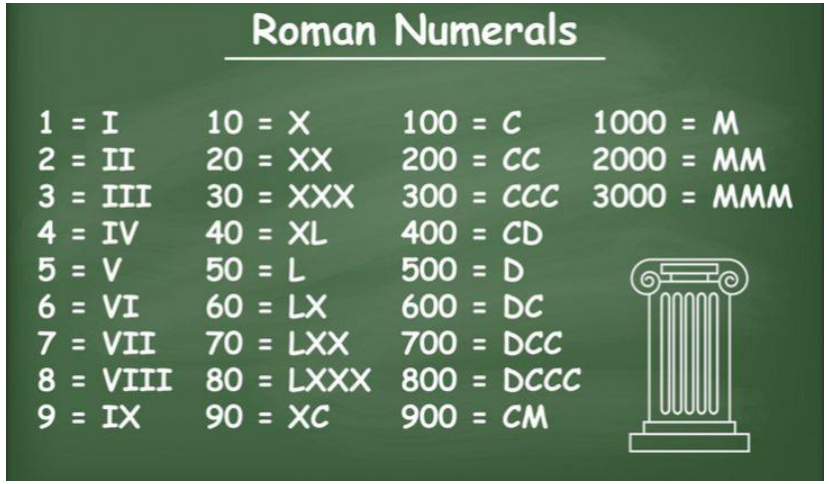
TCSS 343 – Design and Analysis of Algorithms

Algorithm Analysis

Roman Numerals

- Before the Indian numerals were adopted most Europeans used Roman numerals



- How do you do arithmetic with numbers written in Roman Numerals?
  - What is $XLVII + LIII$?
  - What is $V \cdot X$?

Muhammad ibn Musa al-Khwarizmi

- al-Khwarizmi (c. 780 – c. 850) was a Persian scholar who wrote textbooks on how to do arithmetic and algebra using the Indian number system.
  - He introduced the positioning of the decimal.
  - He described the basic steps for adding, subtracting, multiplying and dividing numbers.
  - He developed basic rules of algebra (*al-jabr* means restoration in Arabic).
  - He solved the roots of quadratic polynomials and calculated square roots and digits of π.
- These textbooks were translated into Latin in 1145 and popularized in Europe by Fibonacci in the 15th century.
  - al-Khwarizmi when translated into Latin was *algorithmi*, and his name was used as a root of the word *algorithm* in many European languages.



Algorithm and Problems

- What is an algorithm?
  - An algorithm attempts to solve a problem.

- In computer science a problem is stated as a relationship between input conditions and output expectations.

Addition
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: $N + M$

Subtraction
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: $N - M$

Multiplication
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: NM

Division
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: $Q = N/M$ and $R = N\%M$

- The algorithm is a sequence of steps that transforms the input into the output.

Instances

- An input to a problem that matches the input conditions is called an instance of the problem.

Addition
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: $N + M$

Subtraction
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: $N - M$

Multiplication
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: NM

Division
Input: $N, \mathrm{M} \in \mathbb{Z}$
Output: $Q = N/M$ and $R = N\%M$

- Here are some example instances of the problems above:

$$N = 42, M = 1337$$
$$N = -35, M = 35$$
$$N = 65536, M = 1024$$

Algorithms

- An algorithm is a solution to a problem.
  - Problems and algorithms are abstract.
    - A problem is an abstraction and an instance is a concrete.
    - An algorithm is an abstraction and a program solving the instance is a concrete.
  - An algorithm is stated in pseudocode – a hybrid of natural language and code.
    - The intended audience of an algorithm is other human beings so it is written so that others can understand.
    - The intended audience of a program is the computer and that is why we must speak its language.
  - The algorithm describes the steps a program must take to solve the problem.
- Algorithms are measured by two standards:
  - Correctness: We prefer to demonstrate an algorithm is correct using a formal proof of correctness. When this is not possible we might rely on empirical evidence of an algorithm's performance.
  - Efficiency: We demonstrate an algorithm's efficiency by providing a runtime analysis or analysis of other resource costs. formal runtime analyses often include proofs.

Analysis of Algorithms

- What makes a good algorithm?
  - Correctness
    - An algorithm is always correct.
    - An approximation algorithm is always close to correct according to some measure.
    - A heuristic is often correct but not on all cases.
    - A randomized algorithm has a probability of correctness.
    - We demonstrate these properties with a proof of correctness.
  - Efficiency
    - An algorithm takes some number of steps.
    - An algorithm uses some memory to carry out the steps.
    - An algorithm may use other resources like a communication channel or randomness.
    - We demonstrate the efficiency with quantitative resource analysis.

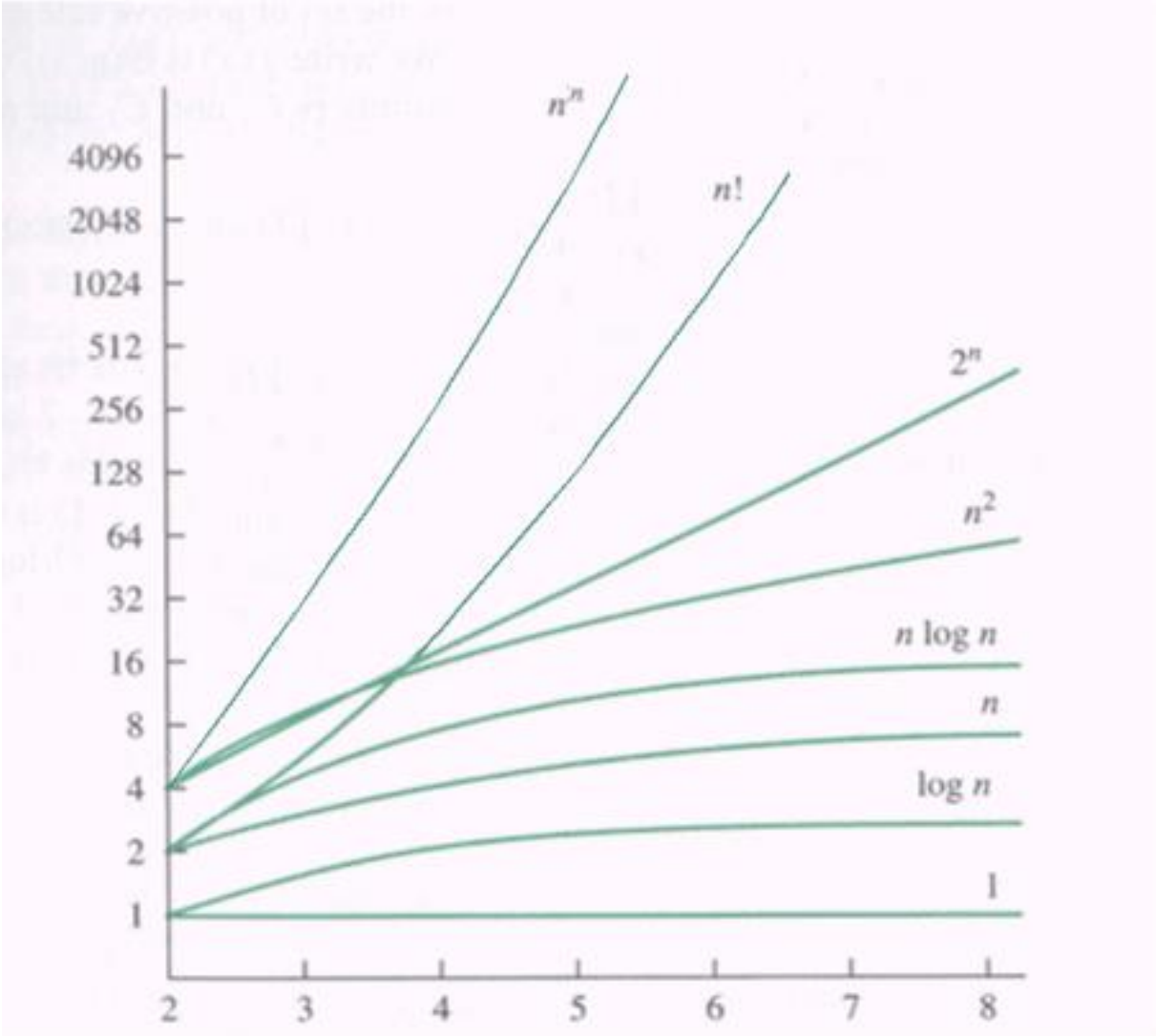
Runtime Functions

- The number of operations an algorithm takes will usually depend on the size of the instance.
  - It is very common that larger inputs take more operations.
- We measure the number of operations an algorithm takes as a function of its input size.
  - For instance this function might express a runtime

$$f(n) = 4n^2 + 6n + 3$$

  - This means that when the algorithm was run on an input of size n it would take $f(n)$ number of operations.
- Not every function expresses a runtime and there are a class of functions that are commonly found in runtime analysis.
  - Runtime functions are typically non-negative and non-decreasing functions.
- Here are some common runtime functions in order of increasing asymptotic growth.
  - Logarithmic
    - $(\log_a n)^d$ for $a, d \in \mathbb{R}^+$
  - Polynomial
    - n^d for $d \in \mathbb{R}^+$
  - Exponential
    - b^n for $b \in \mathbb{R}^+, b \geq 1$
  - Beyond
    - $n!$
    - n^n

Runtime Functions



Input Sizes

- How big is the integer $N = 1000000$?
  - It is common to assume the size of a number is the number itself.
  - However we want to know how large the number is when we write it down in binary.
  - In this case $N = 1000000$ is 7 digits or 20 bits.
  - When considering the size of an integer we will use
$$n = \lfloor \log_2(N) \rfloor + 1$$
  - the number of bits it takes to write the integer.

- With different types of input the size is measured in different ways.
  - Numbers are measured by the number of bits.
  - Strings are measured by the number of characters.
  - Lists and sets are measured by the number of entries.
  - Trees and graphs are measured by the number of nodes.

Asymptotic Notation

- Asymptotic notation, also known as "big-Oh" notation, is a method of grouping similar functions together in sets based on their asymptotic growth rate.
  - The asymptotic growth rate of a function is its rate of growth as n approaches infinity.

- Asymptotic notation is used to refer to a set of functions.
  - The notation $O(f(n))$ means the set of all functions that grow asymptotically no faster than $f(n)$ as n gets large.
  - The notation $\Omega(f(n))$ means the set of all functions that grow asymptotically no slower than $f(n)$ as n gets large.
  - The notation $\Theta(f(n))$ means the set of all functions that grow asymptotically at the same rate as $f(n)$ as n gets large.

Asymptotic Notation

- Each of the sets can be formally defined.
  - Definition 1: A function $f(n) \in O(g(n))$ if there exists positive constants b and n_f such that $f(n) \leq b \cdot g(n)$ for all $n > n_f$.
  - Definition 2: A function $f(n) \in \Omega(g(n))$ if there exists positive constants a and n_f such that $a \cdot g(n) \leq f(n)$ for all $n > n_f$.
  - Definition 3: A function $f(n) \in \Theta(g(n))$ if there exists positive constants a, b and n_f such that $a \cdot g(n) \leq f(n) \leq b \cdot g(n)$ for all $n > n_f$.

- We can use the formal definition to write a proof.

  Theorem: $f(n) = 4n^2 + 6n + 3 \in \Theta(n^2)$

  Proof: To show that $f(n) \in \Theta(n^2)$ by definition we must show there exist three positive constants such that $a \cdot n^2 \leq f(n) \leq b \cdot n^2$ for all $n > n_0$.

  first consider for all integers $n \geq 1$ we know that

$$4n^2 + 6n + 3 \leq 4n^2 + 6n^2 + 3n^2 = 13n^2$$

  Second notice that for all integers $n \geq 1$ we know that

$$4n^2 \leq 4n^2 + 6n + 3$$

  So if we select $a = 4, b = 13$ and $n_0 = 1$ we have shown the definition holds.

Exercise: Asymptotic Notation

- Can we use the definitions to show these relationships hold?
  - $6n^3 + 3n^2 - 14n + 2 \in \Theta(n^3)$
  - $(2n^2 - 4n + 3)^6 \in \Theta(n^{12})$
  - $\log(3n^2 + n - 5) \in \Theta(\log n)$
  - Show that for any two functions $f(n), g(n) \in O(n)$ that $f(n) + g(n) \in O(n)$.
  - Show that $n^{\sin n} \in O(n)$.
  - Is $n^{\sin n} \in \Theta(n)$?

Little-Oh Notation

- The next topic uses "little-oh" notation which is another asymptotic notation not as commonly used.
  - The notation $o(f(n))$ means the set of all functions that grow asymptotically strictly slower than $f(n)$ as n gets large.
  - The notation $\omega(f(n))$ means the set of all functions that grow asymptotically strictly faster than $f(n)$ as n gets large.

- For asymptotic growth rates use the following cheat sheet:
  - $f(n) \in O(g(n))$ means $f(n) \leq g(n)$ in terms of growth rate.
  - $f(n) \in o(g(n))$ means $f(n) < g(n)$ in terms of growth rate.
  - $f(n) \in \Omega(g(n))$ means $f(n) \geq g(n)$ in terms of growth rate.
  - $f(n) \in \omega(g(n))$ means $f(n) > g(n)$ in terms of growth rate.
  - $f(n) \in \Theta(g(n))$ means $f(n) = g(n)$ in terms of growth rate.

The Limit Test

Theorem: If $f(n)$ and $g(n)$ are two runtime functions then

1. If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$ then $f(n) \in \omega\big(g(n)\big)$.
2. If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ then $f(n) \in o\big(g(n)\big)$.
3. If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$ for some non-zero c then $f(n) \in \Theta\big(g(n)\big)$.

Proof sketch:

If the ratio between $f(n)$ and $g(n)$ goes to infinity then $f(n)$ must grow much faster (more than a constant factor) than $g(n)$.

If the ratio between $f(n)$ and $g(n)$ goes to zero then $f(n)$ must grow much slower (more than a constant factor) than $g(n)$.

If the ratio between $f(n)$ and $g(n)$ goes to a constant then $f(n)$ must grow at the same rate as $g(n)$ except for a constant factor.

Example: The Limit Test

Theorem: $6n + 6 \in \Theta(n)$

Proof: We will calculate the limit of $\frac{6n+6}{n}$

$$\lim_{n\to\infty} \frac{6n+6}{n} = \lim_{n\to\infty} \frac{6n}{n} + \frac{6}{n}$$

$$= \lim_{n\to\infty} 6 + \frac{6}{n}$$

$$= \lim_{n\to\infty} 6 + 0$$

$$= 6$$

Since the limit is a non-zero constant we can conclude by the Limit test $6n + 6 \in \Theta(n)$. □

Example: The Limit Test

Theorem: $\log n \in o(n)$

Proof: We will calculate the limit of $\frac{\log n}{n}$

$$\lim_{n\to\infty} \frac{\log n}{n}$$

$$= \lim_{n\to\infty} \frac{\frac{c}{n}}{1} = \lim_{n\to\infty} \frac{c}{n} = 0$$

In the first step we use L'Hopital's rule to simplify the limit.
Since the limit is 0 we can conclude that $\log n \in o(n)$. □

Exercise: The Limit Test

- Put this set of functions in order of asymptotic growth using the limit test. Some may need simplification before applying the limit test. All logarithms are base 2.
  - $(3n^2 + 9)^2$
  - $(n^2 - 3n)\log n$
  - $\frac{n}{2} + 11$
  - $8^{\log n}$
  - $\log(4n + 9)$
  - $2^{\log n^2}$
  - $\frac{n^4 - 2n^3 + 5}{\log n}$

TCSS 343 – Design and Analysis of Algorithms

Simple Iterative Analysis

Loop Analysis

- When determining the number of operations in iterative algorithms we use summations to express the cost of looping structures.
  - How many times does this loop call output?

```
for i from 1 to n do:
    Output i.
```

$$\sum_{i=1}^{n} 1$$

- To do this we must identify relevant cost of the loop.
  - How many iterations of the loop?
    - If the loop has a counter this can be easy.
    - If the loop uses some condition we must attempt to determine when that condition will hold.
    - Summations in math must be indexed by an increasing index.
  - What is the cost of each iteration of the loop?
    - We express the cost relative to the index if the cost changes from iteration to iteration.

Binary Addition

```
Addition
Input:  N, M ∈ ℤ
Output: N + M
```

```
Add(N, M)
    Let c = 0.
    Let n = ⌊log(max{N, M})⌋ + 1.
    for i = 1 to n:                          // Assume LSB first
        Let S_i = (N_i ⊕ M_i) ⊕ c.
        Let c = (N_i ∧ M_i) ∨ (N_i ∧ c) ∨ (M_i ∧ c).
    End for
    Let S_{n+1} = c.
    Return S.
End Add
```

- This is one algorithm for adding integers. We have two questions about this algorithm:
  - Is this algorithm correct?
  - How many logical operations does this algorithm use?

Correctness

- Is this algorithm correct?
  - We can determine this by checking the truth tables.

N_i	M_i	c	$N_i \oplus M_i$	$(N_i \oplus M_i) \oplus c = S_i$	$N_i \wedge M_i$	$N_i \wedge c$	$M_i \wedge c$	$(N_i \wedge M_i) \vee (N_i \wedge c) \vee (M_i \wedge c) = \mathbf{c}$
0	0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0
0	1	0	1	1	0	0	0	0
0	1	1	1	0	0	0	1	1
1	0	0	1	1	0	0	0	0
1	0	1	1	0	0	1	0	1
1	1	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	1

Operations

- How many logical operations does this algorithm use?

```
Add(N, M)
    Let c = 0.
    Let n = ⌊log(max{N, M})⌋ + 1.
    for i = 1 to n:                                  // Assume LSB first
        Let S_i = (N_i ⊕ M_i) ⊕ c.
        Let c = (N_i ∨ M_i) ∧ (N_i ∨ c) ∧ (M_i ∨ c).
    End for
    Let S_{n+1} = c.
    Return S.
End Add
```

- How many times does the loop iterate?
  - n
- How many logical operations are carried out each step?
  - There are 7 logical operators, but it is not clear how many operations each operator takes. So we will select a constant c to represent the cost.
- Can we express the number of logical operations as a summation?

$$\sum_{i=1}^{n} c = c \sum_{i=1}^{n} 1 = cn \in \Theta(n)$$

Binary Multiplication

```
Multiplication
Input:  N, M ∈ ℤ
Output: NM
```

- The common way to multiply is to use the grade school algorithm.

```
Multiply(N, M)
    Let P be an empty list of integers.
    Let n = ⌊log(max{N, M})⌋ + 1.
    for i = 0 to n − 1:                          // Assume LSB first
        If N_i = 1
            Let P_i = M · 2^i.
        Else
            Let P_i = 0.
    End for
    Return ∑_{i=0}^{n−1} P_i.
End Multiply
```

- This is one algorithm for multiplying integers. (There are several others.)
  - We know this algorithm is correct from grade school.
  - How many bit operations does this algorithm use?

Operations

- How many bit operations does the grade school algorithm take?

```
Multiply(N, M)
      Let P be an empty list of integers.
      Let n = ⌊log(max{N, M})⌋ + 1.
      for i = 0 to n − 1:                                  // Assume LSB first
                If N_i = 1
                          Let P_i = M_i · 2^i.
                Else
                          Let P_i = 0.
      End for
      Return Σ_{i=0}^{n−1} P_i.
End Multiply
```

- We will analyze the worst-case runtime of the algorithm in three steps.
  - The first 2 lines take c operations.
  - The loop takes:

$$\sum_{i=0}^{n-1}(n+i)$$

  - The return step takes:

$$\sum_{i=1}^{n-1}(n+i+1)$$

- So, the algorithm takes $c + \sum_{i=0}^{n-1}(n+i) + \sum_{i=1}^{n-1}(n+i+1)$.

Operations

$$c + \sum_{i=0}^{n-1} (n+i) + \sum_{i=1}^{n-1} (n+i+1)$$

$$= c + n^2 + \frac{n(n-1)}{2} + \sum_{i=1}^{n-1} n + \sum_{i=1}^{n-1} i + \sum_{i=1}^{n-1} 1$$

$$= c + n^2 + \frac{n^2}{2} - \frac{n}{2} + n(n-1) + \frac{n(n-1)}{2} + (n-1)$$

$$= c + n^2 + \frac{n^2}{2} - \frac{n}{2} + n^2 - n + \frac{n^2}{2} - \frac{n}{2} + n - 1$$

$$= 3n^2 - n + c - 1$$

$$\in \Theta(n^2)$$

TCSS 343 – Design and Analysis of Algorithms

Advanced Iterative Analysis

Example: Selection Sort

- Consider the algorithm for the Sort problem called selection sort.

```
SelectionSort(A[1 … n])
        Let C be an empty list.
        Repeat until A is empty:
                Let c be the minimum element in A and remove it from the list.
                Add c to the end of C.
        Return C.
End SelectionSort
```

- We need to identify the number of operations executed by the loop.
  - How many iterations in the loop?
    - One element of A is removed each iteration and A has n elements.
  - What is the cost of each iteration?
    - Each iteration has a few steps to consider:
      - Checking if A is empty.
      - Finding the minimum element.
      - Removing the minimum element.
      - Adding the element to the new list.

$\Theta(1)$

Depends on the data structure!
- List of size k - $\Theta(k)$
- Heap of size k - $\Theta(\log k)$

Example: Selection Sort

```
Repeat until A is empty:
            Let c be the minimum element in A and remove it from the list.
            Add c to the end of C.
```

- Putting all this information together we can write a summation to express the number of operations executed by the loop.
  - Note: The list A starts with n elements and ends empty. The function $n - i + 1$ expresses the number of elements in A during iteration i.

$$\sum_{i=1}^{n} \Theta(1) + \Theta(n - i + 1)$$

  - This notation is not exactly correct (though some computer scientists will use this notation). The asymptotic terms don't belong here. Remember they are sets. So we need to select representative functions from these sets to replace these terms.

$$\sum_{i=1}^{n} [c + d(n - i + 1)]$$

  - I have selected the constants c and d to express the unknown constants we know exist from the definition of Θ.

Summation Simplification

$$\sum_{i=1}^{n}[c + d(n - i + 1)]$$

$$= \sum_{i=1}^{n} c + \sum_{i=1}^{n} d(n - i + 1)$$

- Note: We could distribute further on the second sum but will use another trick in a moment.

$$= c\sum_{i=1}^{n} 1 + d\sum_{i=1}^{n}(n - i + 1)$$

$$= cn + d\sum_{i=1}^{n}(n - i + 1)$$

$$= cn + d\sum_{i=1}^{n} i$$

$$= cn + \frac{dn(n+1)}{2} = \frac{dn^2 + (d + 2c)n}{2} \in \Theta(n^2)$$

Heap Sort

- Consider the variant of selection sort that uses a min-heap instead of an unsorted list.
  - Recall: Finding and removing the minimum element takes $\Theta(\log k)$ with a min-heap instead of $\Theta(k)$.

- Substituting in with a function from $\Theta(\log k)$ we get:

$$\sum_{i=1}^{n} [c + d \log(n - i + 1)]$$

$$= \sum_{i=1}^{n} c + \sum_{i=1}^{n} d \log(n - i + 1)$$

$$= c \sum_{i=1}^{n} 1 + d \sum_{i=1}^{n} \log(n - i + 1)$$

$$= cn + d \sum_{i=1}^{n} \log i$$

Binding Sums

- Notice the reordering trick was really helpful because there is no way to distribute the sum over the log as it was. Even with this simplification we need to solve this sum.

$$\sum_{i=1}^{n} \log i$$

- Sums can be very difficult to evaluate exactly.

$$\sum_{i=1}^{n} \log i = f(n)$$

- Luckily when we are pursuing an asymptotic bound we only need to find upper bounds and lower bounds on the function.

$$l(n) \leq \sum_{i=1}^{n} \log i \leq u(n)$$

- If we can find $l(n)$ and $u(n)$ such that they are both in $\Theta\big(g(n)\big)$ then we can use the squeeze theorem to conclude that the sum is also in $\Theta\big(g(n)\big)$.

Upper Bounds

$$\sum_{i=1}^{n} \log i \leq u(n)$$

- We will use the technique of binding the term to find an upper bound.
- We rely on the fact that $1 \leq i \leq n$ and that $\log n$ is increasing to note that

$$\log i \leq \log n$$

- and so

$$\sum_{i=1}^{n} \log i \leq \sum_{i=1}^{n} \log n$$

$$= \log n \sum_{i=1}^{n} 1 = n \log n$$

$$\in \Theta(n \log n)$$

- The binding the term technique is an approximation and will not always give us the correct answer. However it is typically useful in finding upper bounds.

Lower Bounds

$$l(n) \leq \sum_{i=1}^{n} \log i$$

- We will try to use the technique of binding the term to find a lower bound.
- We rely on the fact that $1 \leq i \leq n$ and that $\log n$ is increasing to note that

$$\log 1 \leq \log i$$

- and so

$$\sum_{i=1}^{n} \log i \geq \sum_{i=1}^{n} \log 1$$

$$= 0$$

$$\in \Theta(1)$$

- This technique has correctly found a lower bound. However 0 is a trivial and terrible lower bound.
  - If we want a tight bound we need to find a lower bound that is in $\Theta(n \log n)$.

Lower Bounds

$$l(n) \leq \sum_{i=1}^{n} \log i$$

- We will use the technique of splitting the sum before we use binding the term to find a tight lower bound.
- Splitting the sum begins by re-writing the sum

$$\sum_{i=1}^{n} \log i = \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \log i + \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^{n} \log i$$

- Now we note that the whole sum is greater than half of the sum (since all terms are positive).

$$\sum_{i=1}^{n} \log i \geq \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^{n} \log i$$

- We can now use the technique of binding the term on this sum.

Lower Bounds

$$\sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^{n} \log i \geq \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^{n} \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right)$$

$$= \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^{n} 1$$

- Before we tackle the logarithm lets look at the remaining sum closely.

$$\sum_{i=a}^{b} 1 = b - a + 1$$

$$\sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^{n} 1 = n - \left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + 1 = n - \left\lfloor \frac{n}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil$$

Lower Bounds

$$\left\lceil \frac{n}{2} \right\rceil \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right)$$

- Let's get rid of the ceilings and floors first. We will use the following rules.

$$x - 1 \leq \lfloor x \rfloor \leq x$$
$$x \leq \lceil x \rceil \leq x + 1$$

- So applying this to our function we get

$$\left\lceil \frac{n}{2} \right\rceil \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) \geq \frac{n}{2} \log\left(\frac{n}{2}\right)$$

$$\geq \frac{n}{2}(\log n - \log 2) = \frac{n}{2}(\log n - 1)$$

$$= \frac{n \log n}{2} - \frac{n}{2}$$

$$\in \Theta(n \log n)$$

Exercises

- Use binding the term to find upper bounds for these sums:

$$\sum_{i=1}^{n} \sqrt{i}$$

$$\sum_{i=1}^{n} i^2$$

$$\sum_{i=1}^{n} i \log i$$

- Now use splitting the sum and binding the term to find tight lower bounds for the sums.

TCSS 343 – Design and Analysis of Algorithms

Approximation by Integrals

Approximation by Integrals

- If the summation has the form

$$\sum_{i=k}^{n} f(i)$$

- where $f(i)$ is non-decreasing on the interval $[k, n]$ we can approximate the summation by two definite integrals:

$$\int_{k-1}^{n} f(x)dx \leq \sum_{i=k}^{n} f(i) \leq \int_{k}^{n+1} f(x)dx$$

- Most runtime functions are non-decreasing but if instead we have a non-increasing function we swap the integrals:

$$\int_{k}^{n+1} f(x)dx \leq \sum_{i=k}^{n} f(i) \leq \int_{k-1}^{n} f(x)dx$$

Approximation by Integrals

- We can use integration to approximate this summation:

$$\sum_{i=1}^{n} i^2$$

- Upper bound:

$$\sum_{i=1}^{n} i^2 \leq \int_{1}^{n+1} x^2 dx$$

$$= \frac{(n+1)^3}{3} + c \; - \frac{1^3}{3} - c$$

$$= \frac{n^3 + 3n^2 + 3n + 1}{3} - \frac{1}{3}$$

$$= \frac{n^3}{3} + n^2 + n$$

- Notice that this upper bound is much tighter than the upper bound we find using the binding the term method.

$$\sum_{i=1}^{n} i^2 \leq n^3$$

Approximation by Integrals

- We can use integration to approximate this summation:

$$\sum_{i=1}^{n} i^2$$

- Lower bound:

$$\sum_{i=1}^{n} i^2 \geq \int_0^n x^2 dx$$

$$= \frac{n^3}{3} + c \; - \frac{0^3}{3} - c$$

$$= \frac{n^3}{3}$$

- Notice that this lower bound is much tighter than the lower bound we find using the splitting the sum and binding the term method.

$$\sum_{i=1}^{n} i^2 \geq \frac{n^3}{8}$$

Approximating Sums

- We have seen two methods to approximate sums:
  - Binding the term and splitting the sum.
  - Approximation by integration.
- Approximation by integrals is a much more accurate approximation method. We can see this by looking at:

$$\sum_{i=1}^{n} i^2$$

- Binding the term and splitting the sum gave us these bounds:

$$\frac{n^3}{8} \leq \sum_{i=1}^{n} i^2 \leq n^3$$

- The approximation by integrals gave us these bounds:

$$\frac{n^3}{3} \leq \sum_{i=1}^{n} i^2 \leq \frac{n^3}{3} + n^2 + n$$

- It is worth noting the closed form for this sum is known:

$$\sum_{i=1}^{n} i^2 = \frac{n^3}{3} + \frac{n^2}{2} + \frac{n}{6}$$